

ECON485 - Homework Assignment 2

SQL Implementation

Part 1a & 1b: Creating Tables and Inserting Data

The following SQL statements implement the 2NF schema from Homework 1 and populate it with sample data.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    FullName VARCHAR(100)  
);  
  
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY,  
    CourseCode VARCHAR(10),  
    Title VARCHAR(100),  
    Credits INT  
);  
  
CREATE TABLE Sections (  
    SectionID INT PRIMARY KEY,  
    CourseID INT,  
    SectionNumber INT,  
    Capacity INT,  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);  
  
CREATE TABLE Registrations (  
    StudentID INT,  
    SectionID INT,  
    PRIMARY KEY (StudentID, SectionID),  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (SectionID) REFERENCES Sections(SectionID)  
);  
  
-- Example Data Insertions  
INSERT INTO Students (StudentID, FullName) VALUES  
(1, 'Alice Smith'), (2, 'Bob Johnson'), (3, 'Charlie Brown');  
  
INSERT INTO Courses (CourseID, CourseCode, Title, Credits) VALUES  
(101, 'ECON101', 'Intro to Microeconomics', 3),  
(102, 'MATH101', 'Calculus I', 4),  
(103, 'ECON201', 'Intermediate Microeconomics', 3);  
  
INSERT INTO Sections (SectionID, CourseID, SectionNumber, Capacity) VALUES  
(1, 101, 1, 50), (2, 102, 1, 40), (3, 103, 1, 30);  
  
INSERT INTO Registrations (StudentID, SectionID) VALUES  
(1, 1), (1, 2), (2, 1), (3, 3);
```

Part 1c: Demonstrate Add and Drop

```
-- Add Action: Student 2 registers for Section 2
INSERT INTO Registrations (StudentID, SectionID) VALUES (2, 2);

-- Drop Action: Student 1 drops Section 2
DELETE FROM Registrations WHERE StudentID = 1 AND SectionID = 2;
```

Part 2a & 2b: Prerequisites and Assistive SQL

To handle prerequisites, two new tables are introduced: Prerequisites and CompletedCourses. Below are the queries the application uses to validate rules.

```
CREATE TABLE Prerequisites (  
    PrerequisiteID INT PRIMARY KEY,  
    CourseID INT,  
    RequiredCourseID INT,  
    MinimumGrade DECIMAL(3,2),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID),  
    FOREIGN KEY (RequiredCourseID) REFERENCES Courses(CourseID)  
);
```

```
CREATE TABLE CompletedCourses (  
    CompletionID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    Grade DECIMAL(3,2),  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)  
);
```

```
-- Query 1: Find all prerequisites of a course (e.g., target CourseID = 103)  
SELECT RequiredCourseID, MinimumGrade  
FROM Prerequisites  
WHERE CourseID = 103;
```

```
-- Query 2: Check if student (e.g., StudentID = 3) meets prerequisites for  
CourseID = 103  
SELECT  
    p.RequiredCourseID,  
    c.Grade AS StudentGrade,  
    p.MinimumGrade,  
    CASE  
        WHEN c.Grade >= p.MinimumGrade THEN 'Eligible'  
        ELSE 'Not Eligible'  
    END AS Status  
FROM Prerequisites p  
LEFT JOIN CompletedCourses c  
    ON p.RequiredCourseID = c.CourseID AND c.StudentID = 3  
WHERE p.CourseID = 103;
```